📓 جزوه سیستم عامل - فصل ۷: مثال‌های همگامی

(بخش پنجم و پایانی واقعی: رویکردهای جایگزین و نوین)

۱. چرا به رویکردهای جدید نیاز داریم؟

با ظهور پردازنده‌های چند هسته‌ای (Multicore)، برنامه‌نویسی همروند اجتناب‌ناپذیر شده است. اما ابزارهای قدیمی مثل **Mutex** و **Semaphore** دو مشکل بزرگ دارند:

1. **خطر بن‌بست (Deadlock):** اگر ترتیب قفل‌ها را رعایت نکنید، برنامه قفل می‌شود.
2. **شرایط مسابقه (Race Condition):** اگر قفل‌گذاری را فراموش کنید، داده‌ها خراب می‌شوند.
3. **توسعه‌پذیری (Scalability):** با زیاد شدن تعداد نخ‌ها، مدیریت قفل‌ها بسیار دشوار و کند می‌شود.

بنابراین، محققان روش‌های جدیدی ابداع کرده‌اند که در ادامه می‌خوانیم.

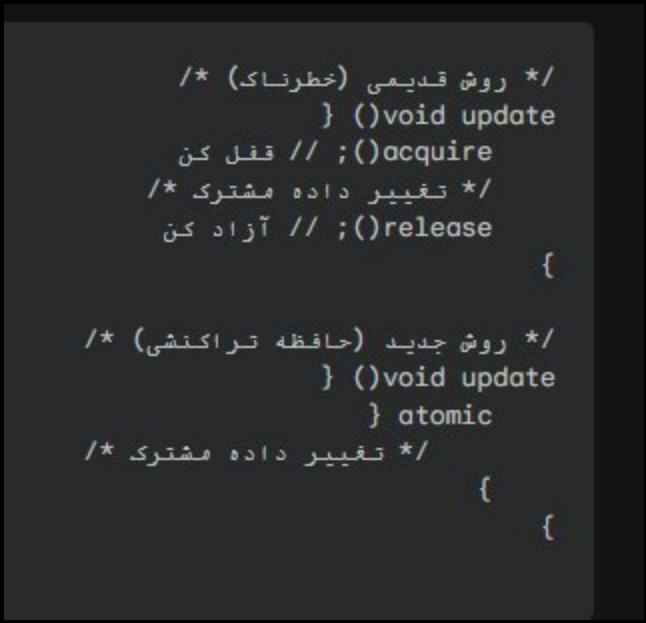
۲. حافظه تراکنشی (Transactional Memory - TM)

این ایده از **پایگاه داده‌ها (Databases)** گرفته شده است. در دیتابیس، یک تراکنش (Transaction) مجموعه‌ای از تغییرات است که یا **همه با هم انجام می‌شوند (Commit)** یا **هیچ‌کدام انجام نمی‌شوند (Rollback)**.

نحوه عملکرد:

به جای استفاده از lock() و unlock()، کد حساس را در یک بلوک atomic می‌گذاریم. سیستم خودش تضمین می‌کند که این بلوک به صورت اتمیک اجرا شود.

مقایسه کد قدیم و جدید:



مزایای TM:

1. **مسئولیت با سیستم است:** برنامه‌نویس نگران قفل‌گذاری نیست.
2. **بدون بن‌بست:** چون قفلی وجود ندارد، بن‌بستی هم رخ نمی‌دهد.
3. **همروندی بالا:** سیستم خودش تشخیص می‌دهد کدام دستورات می‌توانند همزمان اجرا شوند (مثلاً خواندن‌های همزمان).

انواع پیاده‌سازی TM:

1. **حافظه تراکنشی نرم‌افزاری (STM):** توسط کامپایلر و کتابخانه‌ها مدیریت می‌شود. (کندتر است چون کد اضافی تزریق می‌کند).
2. **حافظه تراکنشی سخت‌افزاری (HTM):** توسط سخت‌افزار (کش پردازنده و پروتکل‌های حافظه) مدیریت می‌شود. (سریع‌تر است ولی نیاز به پردازنده‌های جدید دارد).

۳. OpenMP (رابط چندپردازشی باز)

OpenMP مجموعه‌ای از دستورات کامپایلر (Compiler Directives) برای زبان‌های C و C++ است. به جای اینکه دستی نخ بسازید (pthread_create)، به کامپایلر می‌گویید "این قسمت را موازی کن".

دستور #pragma omp critical

این دستور دقیقاً مثل یک **قفل انحصار متقابل (Mutex)** عمل می‌کند ولی بسیار ساده‌تر است.

مثال: فرض کنید متغیر counter مشترک است.

```
} void update(int value)
pragma omp critical#
}
;counter += value // فقط یک نخ در لحظه وارد این بلوک می‌شود
{
{
```

- **مزیت:** استفاده بسیار آسان.
- **عیب:** هنوز هم ممکن است بن‌بست رخ دهد (اگر چندین بخش بحرانی تو در تو داشته باشیم) و هنوز برنامه‌نویس باید تشخیص دهد کجا بحرانی است.

۴. زبان‌های برنامه‌نویسی تابعی (Functional Programming)

زبان‌های رایج مثل C، Java و Python **دستوری (Imperative)** هستند؛ یعنی "حالت" (State) متغیرها مدام تغییر می‌کند.

مشکل اصلی همگامی: تغییر متغیرهای مشترک (Mutable State).

زبان‌های تابعی (مثل **Haskell**، **Scala**، **Erlang**) یک اصل بنیادین دارند: **"متغیرها تغییرناپذیرند (Immutable)."** وقتی متغیری مقدار گرفت، دیگر عوض نمی‌شود.

نتیجه شگفت‌انگیز:

اگر متغیرها تغییر نکنند، پس **شرایط مسابقه (Race Condition)** هم وجود ندارد!

- نیازی به قفل نیست.
- نیازی به سمافور نیست.
- بن‌بست (به شکل سنتی) وجود ندارد. این زبان‌ها برای سیستم‌های توزیع‌شده و همروند ایده‌آل هستند.

۵. 📝 خلاصه جامع فصل ۷ (برای مرور سریع)

این فصل کاربرد ابزارهای همگامی (فصل ۶) را در مسائل واقعی بررسی کرد.

مسائل کلاسیک همگامی:

- **بافر محدود:** ۳ سمافور (mutex, empty, full) برای هماهنگی تولیدکننده/مصرف‌کننده.
- **خوانندگان-نویسندگان:** چالش اولویت‌دهی. (اگر اولویت با خواننده باشد، نویسنده قحطی‌زده می‌شود). از rw_mutex استفاده شد.
- **فلاسفه خوش‌خوراک:** چالش تخصیص منابع (چوبک). راه حل ساده سمافوری باعث **بن‌بست** می‌شود. راه حل **مانیتور** (چک کردن دو همسایه) ایمن است.

همگامی در سیستم‌عامل‌ها:

- **Windows:**
  - تک‌پردازنده: غیرفعال‌سازی وقفه.
  - چندپردازنده: Spinlock (برای زمان کوتاه).
  - سطح کاربر: Dispatcher Objects (Mutex, Semaphore, Event, Timer).
- **Linux:**
  - atomic_t: سریع‌ترین روش (اعداد اتمیک).
  - mutex_lock: اگر قفل نباشد، نخ می‌خوابد.
  - spinlock: اگر قفل نباشد، نخ می‌چرخد (فقط در چندپردازنده).
  - مدیریت preempt_disable برای تک‌پردازنده.

APIهای همگامی:

- **POSIX (Pthreads):** استاندارد C/UNIX. توابع pthread_mutex_lock، sem_wait و pthread_cond_wait.
- **Java:**
  - **Monitors:** کلمه کلیدی synchronized (روش قدیمی و راحت). هر شیء یک Entry Set و یک Wait Set دارد.
  - **java.util.concurrent:** کلاس‌های Semaphore، ReentrantLock و Condition (روش جدید و منعطف).

رویکردهای نوین (آینده همگامی):

- **Transactional Memory:** بلوک‌های atomic (بدون قفل دستی).
- **OpenMP:** مدیریت توسط کامپایلر (#pragma).
- **Functional Languages:** حذف صورت‌مسئله با متغیرهای تغییرناپذیر (Erlang, Scala).

پایان قطعی فصل ۷.